

PRODUCT REQUIREMENTS DOCUMENT

Implementasi Progressive Web App (PWA) & Web Push Notification

Aplikasi NemaFi - Manajemen Pelanggan & Otomatisasi Layanan Internet

PT Ninama Raga Pranala

Dokumen	Product Requirements Document (PRD)
Produk	NemaFi
Fitur	PWA + Web Push Notification
Penulis	Muhammad Mizan Al Mujadid
Versi	1.0
Tanggal	3 Juli 2026
Status	Draft untuk Review

1. Ringkasan Eksekutif

NemaFi adalah aplikasi web manajemen pelanggan dan otomatisasi pengelolaan layanan internet yang sedang dibangun untuk PT Ninama Raga Pranala, sebuah Internet Service Provider (ISP). Aplikasi ini mengintegrasikan manajemen data pelanggan dan billing, kontrol multi-node router MikroTik (via MikroTik API dan SSH), serta otomatisasi isolasi/pemulihan layanan berdasarkan status pembayaran, dengan arsitektur Next.js (frontend), Express.js (backend), Socket.IO (realtime), dan PostgreSQL (basis data).

Dokumen ini mendefinisikan kebutuhan produk untuk penambahan kapabilitas Progressive Web App (PWA) dan Web Push Notification pada NemaFi. Saat ini, notifikasi realtime hanya berjalan melalui Socket.IO ketika dashboard aktif dibuka di browser; begitu tab/browser ditutup, Admin dan Teknisi kehilangan visibilitas terhadap kejadian operasional penting seperti router offline, tagihan jatuh tempo, atau tiket gangguan baru. Perubahan ini bersifat aditif dan modular terhadap arsitektur yang sudah ada, tanpa mengganti komponen inti yang berjalan.

2. Latar Belakang dan Masalah

Berdasarkan proposal perubahan arsitektur yang diajukan, kondisi saat ini memiliki keterbatasan berikut:

- Socket.IO hanya mengirim event ketika dashboard dalam keadaan terbuka di browser aktif.
- Admin/Teknisi tidak menerima pemberitahuan apa pun saat browser atau tab ditutup, meski terjadi insiden penting di jaringan atau proses bisnis (misalnya router offline atau tagihan jatuh tempo).
- Respons operasional terhadap gangguan jaringan dan proses administratif (pembayaran, suspend, reaktivasi) menjadi lambat karena bergantung pada pengguna yang secara aktif membuka dashboard.
- Aplikasi belum dapat diinstal layaknya aplikasi native di perangkat desktop maupun mobile, sehingga aksesibilitas dan keterlibatan pengguna (khususnya teknisi lapangan) masih terbatas.

Kondisi ini selaras dengan permasalahan yang diidentifikasi pada tahap perancangan sistem NemaFi, yaitu keterlambatan eksekusi teknis (isolasi/pemulihan layanan) yang dapat mencapai 1x24 jam karena proses yang belum sepenuhnya realtime dan mandiri dari keberadaan pengguna di depan layar.

3. Tujuan Produk

- Menghadirkan notifikasi lintas perangkat (desktop dan mobile) kepada Admin dan Teknisi, baik saat browser terbuka maupun tertutup.
- Mempertahankan Socket.IO sebagai mekanisme realtime dashboard yang sudah berjalan, tanpa perubahan pada arsitektur inti.
- Menjadikan NemaFi dapat diinstal (installable) sebagai Progressive Web App layaknya aplikasi native di berbagai perangkat.
- Mempercepat waktu respons Admin/Teknisi terhadap kejadian kritis operasional (router offline, tunggakan pembayaran, tiket gangguan baru).

3.1 Metrik Keberhasilan (Success Metrics)

Metrik	Target Indikatif
Delivery rate Web Push ke subscriber aktif	≥ 95%
Waktu dari event terjadi ke notifikasi diterima	< 5 detik
Penurunan waktu eksekusi isolasi/pemulihan layanan	Signifikan dibanding proses manual saat ini (hingga 1x24 jam)
Adopsi instalasi PWA oleh Admin/Teknisi	≥ 80% pengguna internal
Tidak ada regresi pada fitur Socket.IO dashboard existing	0 insiden

4. Target Pengguna

Peran	Kebutuhan terhadap Fitur Ini
Admin	Menerima notifikasi pembayaran berhasil, tagihan jatuh tempo, suspend/reaktivasi otomatis, dan tiket baru, meski dashboard tertutup.
Teknisi	Menerima notifikasi router offline/online, tiket baru yang di-assign, serta hasil instalasi/gangguan, agar dapat merespons cepat di lapangan.
Pelanggan	Di luar cakupan fase ini (lihat Batasan Ruang Lingkup).

5. Arsitektur

5.1 Arsitektur Saat Ini

Komponen	Teknologi
Frontend	Next.js
Backend	Express.js
Realtime	Socket.IO
Basis Data	PostgreSQL
Integrasi Jaringan	MikroTik API, SSH2

5.2 Arsitektur Usulan (Penambahan)

Komponen baru yang ditambahkan bersifat modular di atas arsitektur eksisting:

- manifest.json - deskriptor PWA agar aplikasi dapat diinstal.
- Service Worker (via next-pwa) - menangani caching dan penerimaan push event di sisi klien.
- Web Push API & Notification API (browser) - menampilkan notifikasi native di perangkat pengguna.
- Library web-push pada Express.js - mengirim push message terenkripsi ke browser.
- VAPID Keys - autentikasi identitas server pengirim push notification.
- Tabel push_subscriptions pada PostgreSQL - menyimpan endpoint subscription tiap perangkat pengguna.

5.3 Diagram Arsitektur (Konseptual)

Admin (PWA)

-> *Next.js*

- *Socket.IO (Realtime Dashboard)*

- *Service Worker (Web Push)*

-> *Express.js*

- *REST API*

- *web-push*

- *MikroTik API*

- *SSH2*

- *PostgreSQL*

5.4 Alur Event Notifikasi

Contoh alur untuk kasus router offline:

1. Router terputus dari jaringan (offline).
2. Express.js menerima/mendeteksi event tersebut.
3. Event di-emit melalui Socket.IO ke seluruh dashboard yang sedang aktif (realtime).
4. Secara paralel, Express.js mengirim Web Push ke seluruh perangkat terdaftar milik Admin/Teknisi (endpoint pada push_subscriptions).
5. Notifikasi tetap muncul di perangkat pengguna meskipun browser dalam keadaan tertutup.

6. Kebutuhan Fungsional

6.1 Instalasi Aplikasi (PWA)

- Sistem harus menyediakan manifest.json sehingga NemaFi dapat ditambahkan ke home screen/desktop layaknya aplikasi native.

- Sistem harus mendaftarkan Service Worker melalui next-pwa untuk mendukung caching aset dan penerimaan push event.

6.2 Registrasi Subscription Push

- Pengguna (Admin/Teknisi) harus dapat memberikan izin (permission) notifikasi browser.
- Setelah izin diberikan, sistem menyimpan data subscription (endpoint, p256dh_key, auth_key, user_agent) ke tabel push_subscriptions yang terhubung dengan user_id.
- Sistem harus mendukung banyak subscription per pengguna (multi-perangkat: desktop dan mobile sekaligus).
- Sistem harus dapat menghapus/menonaktifkan subscription yang sudah tidak valid (misalnya browser unsubscribe atau endpoint kedaluwarsa).

6.3 Pengiriman Web Push

- Backend (Express.js) harus dapat mengirim Web Push ke satu atau banyak subscription menggunakan library web-push dengan autentikasi VAPID Keys.
- Pengiriman push harus berjalan paralel dengan emit Socket.IO tanpa saling memblokir (non-blocking).

6.4 Cakupan Event Notifikasi

Fitur ini harus mendukung pengiriman notifikasi untuk event-event berikut:

Event	Penerima
Router offline / online	Teknisi
Pembayaran berhasil	Admin
Tagihan jatuh tempo	Admin
Suspend otomatis (isolasi layanan)	Admin, Teknisi
Reaktivasi layanan	Admin, Teknisi
Tiket baru	Admin
Assign teknisi ke tiket	Teknisi terkait
Hasil instalasi	Admin, Teknisi terkait

6.5 Interaksi Notifikasi

- Klik pada notifikasi harus mengarahkan pengguna ke halaman/dashboard yang relevan dengan event terkait.

7. Kebutuhan Non-Fungsional

Kategori	Kebutuhan
Kompatibilitas	Perubahan tidak boleh mengubah perilaku Socket.IO dan dashboard realtime yang sudah berjalan.
Modularitas	Komponen PWA dan Web Push ditambahkan sebagai lapisan terpisah agar risiko regresi rendah.
Keamanan	VAPID Keys dan kredensial push disimpan sebagai secret/environment variable, tidak di-hardcode di kode sumber.
Reliabilitas	Kegagalan pengiriman push (endpoint kedaluwarsa) tidak boleh mengganggu alur bisnis utama (mis. proses isolasi tetap berjalan).
Kinerja	Pengiriman push massal ke banyak subscription tidak boleh menghambat response time REST API utama.
Skalabilitas	Struktur tabel push_subscriptions mendukung banyak perangkat per pengguna tanpa perubahan skema di masa depan.

8. Perubahan Basis Data

Penambahan tabel baru push_subscriptions pada PostgreSQL:

Kolom	Deskripsi
id	Primary key subscription.
user_id	Referensi ke pengguna (Admin/Teknisi) pemilik subscription.
endpoint	URL endpoint push service dari browser.

Kolom	Deskripsi
p256dh_key	Public key untuk enkripsi payload push.
auth_key	Auth secret untuk enkripsi payload push.
user_agent	Informasi perangkat/browser pemilik subscription.
created_at	Waktu subscription dibuat.
updated_at	Waktu subscription terakhir diperbarui.

9. Teknologi yang Digunakan

Layer	Teknologi
Frontend	Next.js + next-pwa + Service Worker
Backend	Express.js + web-push
Browser API	Push API, Notification API
Basis Data	PostgreSQL
Realtime (tetap dipertahankan)	Socket.IO

10. Tahapan Implementasi

Sprint	Cakupan
Sprint 1	Setup PWA - manifest.json, konfigurasi next-pwa, Service Worker dasar.
Sprint 2	Registrasi subscription - permintaan izin notifikasi, penyimpanan ke tabel push_subscriptions.
Sprint 3	Pengiriman Web Push - integrasi library web-push, VAPID Keys, endpoint pengiriman.
Sprint 4	Integrasi seluruh event notifikasi (router, pembayaran, tiket, dsb.) ke alur push.
Sprint 5	Pengujian menyeluruh - functional testing, uji lintas perangkat/browser, uji regresi Socket.IO.

11. Analisis Dampak

- Tidak mengubah arsitektur utama NemaFi; Socket.IO tetap digunakan sebagai mekanisme realtime dashboard.
- Penambahan bersifat modular sehingga risiko terhadap fitur yang sudah berjalan tergolong rendah.
- Diperlukan penyesuaian minor pada konfigurasi build Next.js untuk mendukung next-pwa dan Service Worker.
- Diperlukan migrasi basis data untuk penambahan tabel push_subscriptions.

12. Batasan Ruang Lingkup (Out of Scope)

- Notifikasi push untuk role Pelanggan tidak termasuk dalam fase ini.
- Notifikasi native mobile (via app store / FCM khusus native app) tidak dicakup; fokus pada Web Push melalui PWA.
- Perubahan pada modul MikroTik API, SSH, dan billing di luar kebutuhan pengiriman event notifikasi tidak termasuk dalam dokumen ini.

13. Pertanyaan Terbuka

- Apakah diperlukan retry mechanism/queue (mis. BullMQ) untuk pengiriman push yang gagal agar tidak hilang begitu saja?
- Berapa lama masa retensi subscription yang tidak aktif sebelum dihapus otomatis dari push_subscriptions?
- Apakah notifikasi memerlukan tingkat prioritas/kategori (mis. kritis vs informatif) yang memengaruhi cara tampil di perangkat?